Open in app ↗ Sign up Sign in

Medium Search

Affirm Tech Blog · Follow publication

Artificial Intelligence + Machine Learning + Data Engineering + Data Science +

Data Architecture +

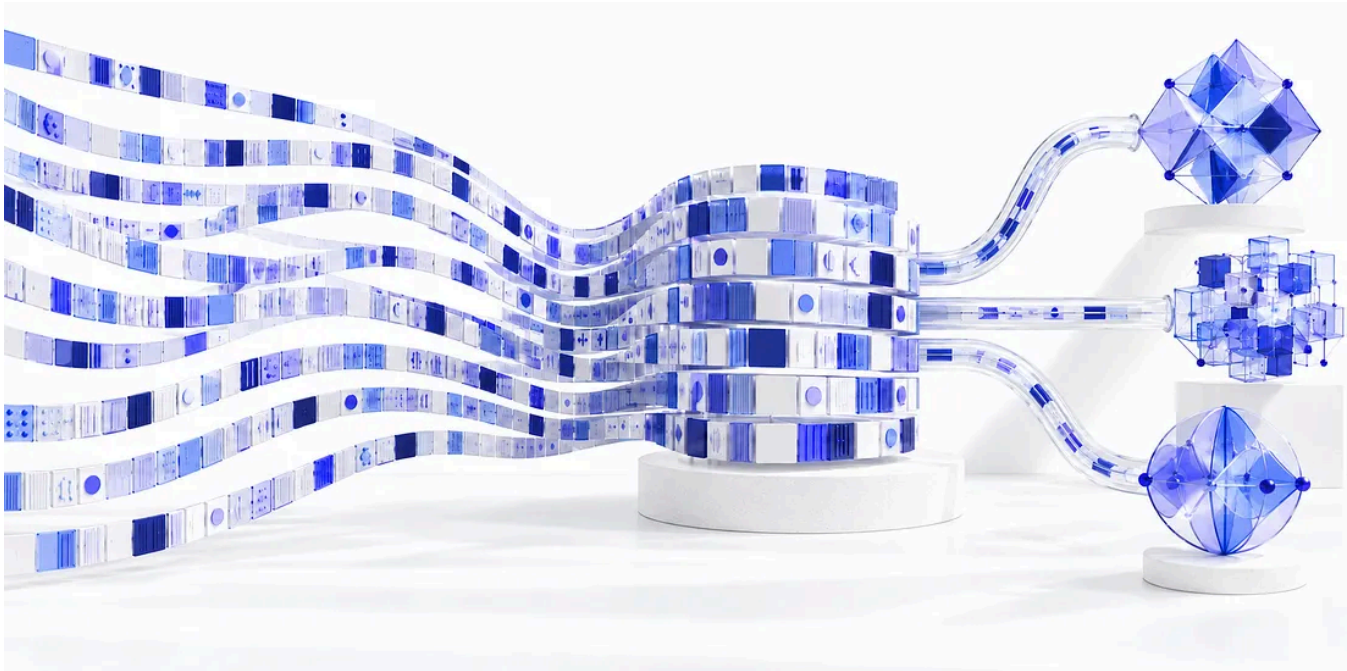
Building Affirm's Intelligence Layer

6 min read · Oct 6, 2026

Affirm Technology Follow

Listen Share

Authors: Yiyang Qian, Aayush Saxena, Shanghao Chen, *Machine Learning and Decision Platform, Affirm*



At checkout, an Affirm model works with the information available then. As a loan progresses, its repayment history adds information that later decisions can use.

Making that experience useful to successive models requires a consistent way to describe it across the systems that record it.

We are building the **Affirm User Data Model**, or **UDM**, as a foundation of Affirm's intelligence layer. It gives teams a governed, shared view of user data, without removing data ownership from the domains that produce it. Its strategic purpose is to make Affirm's history reusable across models and products, including the foundation model work ahead.

The history contains more than the total

Our recent post, Learning Richer Credit Representations with Transformers at Affirm, illustrates why: two consumers each have two credit accounts and two late accounts, duration over six months. One's late payments occurred earlier, on one account; the other's occurred more recently, across two. The counts match. The timing and account relationships differ.

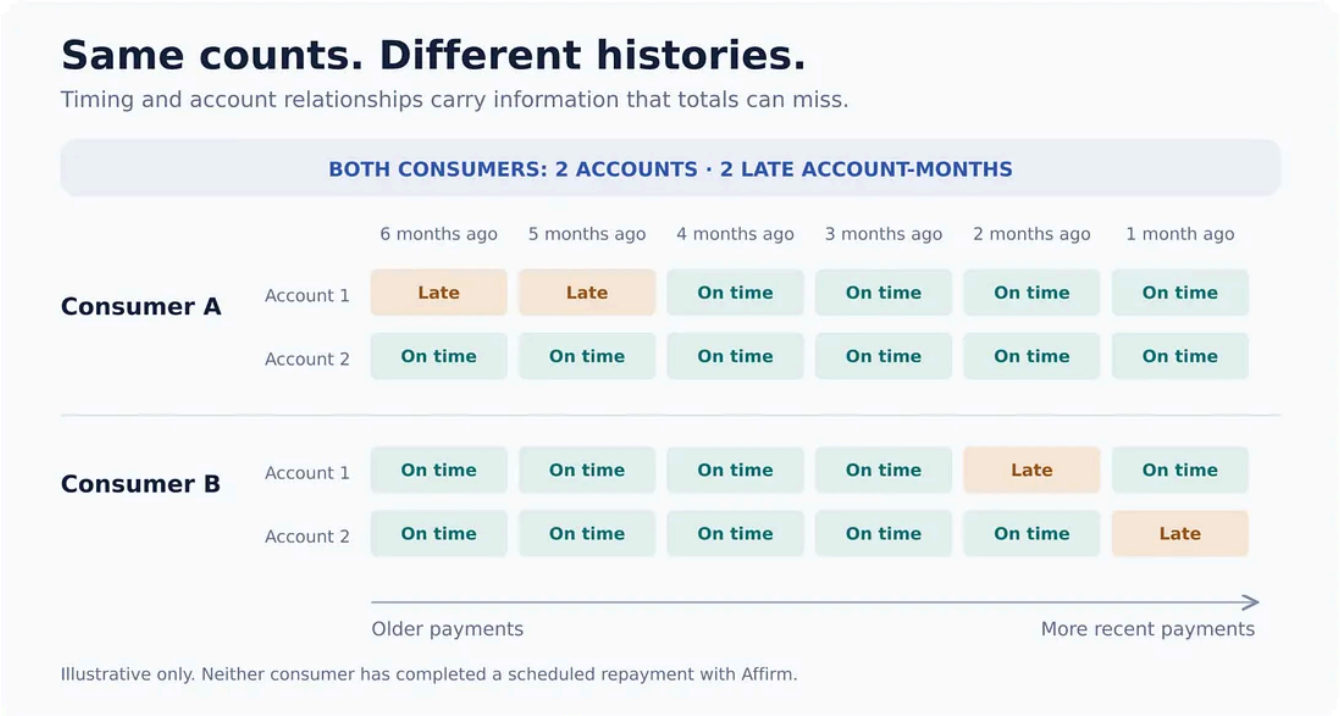


Figure 1. An illustrative credit report example: equal counts can conceal different histories. Neither consumer has completed a scheduled repayment with Affirm; no individual risk ranking is implied.

The Affirm machine learning team built a hybrid transformer model. The model combines sequence-based transformer embeddings with established summary features via AFT XGBoost. The model proved significant business benefits by core metrics.

That result motivates a broader question: One model learned more from a history assembled for it; how much more could every model learn if that history were shared?

Our internal history spans domains. Underwriting decisions, loan state, and repayment activity live in systems with different responsibilities. Learning across them requires consistent definitions and relationships: which loan a payment belongs to, when a state changed, and what information was available for a particular decision. UDM is where those definitions are set once and read by everyone.

From current attributes to event sequences

An attribute describes a current state. An event history preserves what happened over time. Relationships connect that history to the relevant user, loan, or other entity. Together, these let a consumer understand both where a user stands and how they arrived there.

Consider a user with two loans. A current view can describe the latest state of each. A history can preserve earlier state changes and repayment activity, keeping the connection between each record and its loan. That structure matters: each payment stays tied to its own loan, and the information a model could have seen earlier stays available after a later update arrives.

UDM's role is to make these facts available with shared definitions, ownership, and lineage. Downstream model inputs can select the relevant history and arrange it for a particular task. A shared history can support different time windows, input shapes, and model architectures.

This is the representation change we are enabling: from relying on selected summaries to also making structured histories available for learning. **UDM preserves the underlying history; models learn representations from it.** An event sequence is an input. An embedding is one possible learned representation of that input.

The distinction keeps reuse practical. Two teams can share the same history without agreeing on how to encode it. Models keep the choices that are theirs, which slice to read and how to shape it for a given architecture, and UDM supplies the material those choices are made from.

The strategic bet: learning that carries across models

Affirm's bet is on its data. The modeling side is moving from hand-built features to learned representations, and the goal underneath that shift is to capture far more of the intelligence already sitting in our own data while making each new model cheaper to ship than the last.

Data work done once compounds across every model after it. Most of the effort in shipping a model lands before training: locating the inputs, wiring up access, and preparing the history. Until now each model has done that work for itself. Doing it once, in a shared layer, is what lets the second model start where the first finished, and the tenth start further along still. Consistent definitions do the same for meaning: when every domain describes a user the same way, the work done for one consumer carries to the next.

Get Affirm Technology's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

Remember me for faster sign in

Representations are learned from records, so the record comes first. Once what sits underneath is declared, owned, and described the same way across every domain, the representation layer becomes reachable, which is why we are building this layer ahead of the models that will read it. Teams assemble what they need today, and they will keep shipping either way. This is the version where that groundwork is already done for them.

The data is what stays. Serving layers turn over roughly every two years and will again; compute gets bought, features recomputed, models replaced on a cadence. The ordered record of what a user did across domains outlasts all of them, and it is the one item on that list we build rather than buy. Every event we capture and describe well today becomes available to every model that comes after. That is why we are starting now.

Where UDM sits in the intelligence layer

UDM serves a governed replica of data from its producing domains. Those domains remain authoritative. UDM supplies the shared access contract, ownership metadata, lineage, and freshness expectations needed to make their data usable across teams. For authoritative state or domain behavior, the source domain remains the right place to go.

For machine learning, the roles are distinct and stacked: UDM holds the substrate, the Machine Learning Signal Platform serves it onward as features and learned representations, and training and serving turn those into decisions.

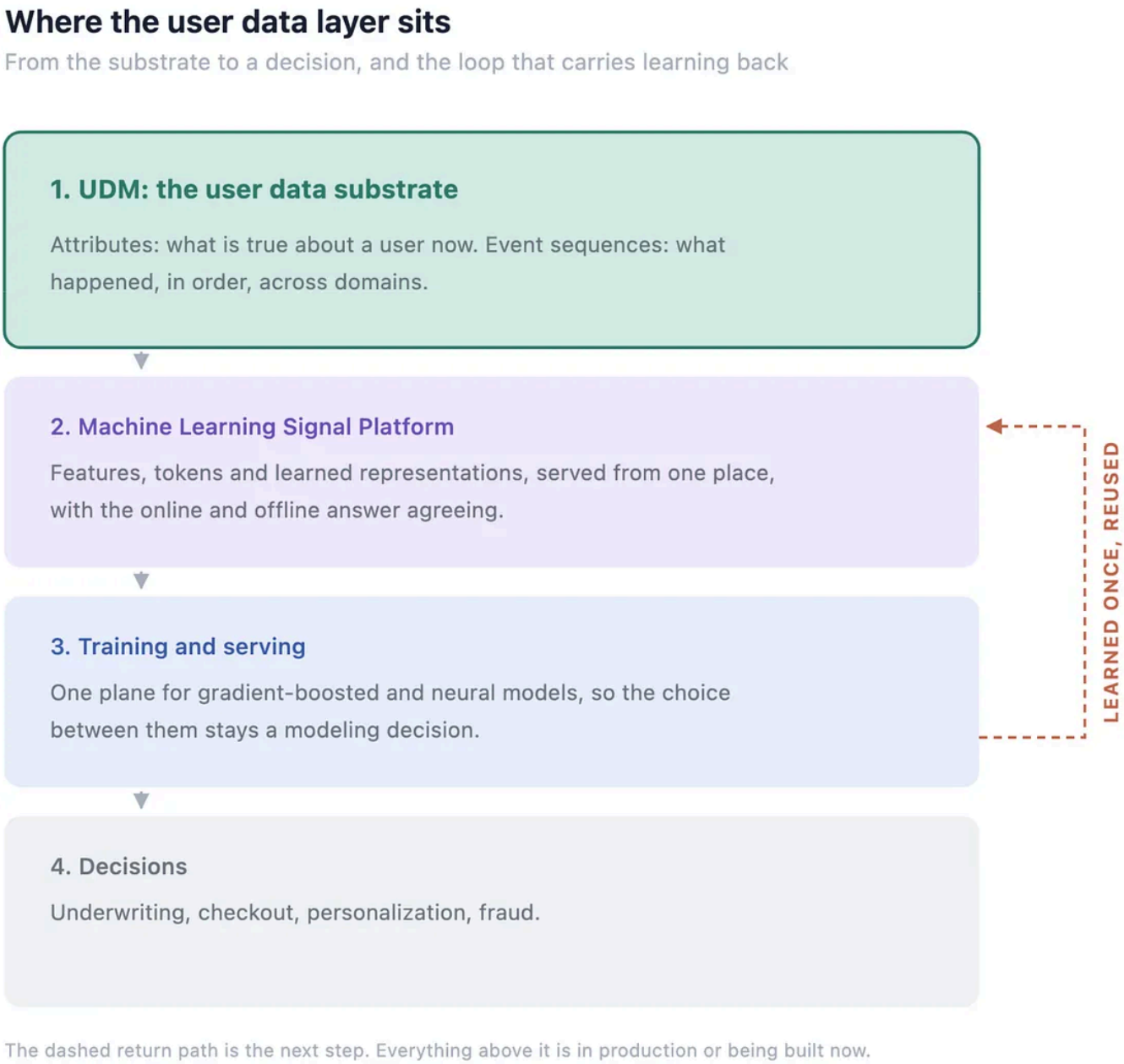


Figure 2. The path from shared user data to a decision. The dashed return path, representations learned in training served back through the same surface, is the next step.

Making shared history dependable

Reuse rests on trust in what you read. UDM's design brings a registry, a low latency read path, and offline data under shared definitions. Online and offline consistency means agreement on semantics and declared time boundaries. The two stores update on different schedules, so each attribute carries an explicit freshness expectation and a reconciliation path.

Three engineering choices make this practical.

- **Quality belongs with the definition.** The same declaration that names an attribute's type, owner, and source also carries its parity, freshness, null, and volume checks.
- **Event histories add a temporal requirement.** Separating when something happened from when it became available is what lets training see only what a decision could have seen; point-in-time reads over shared histories are part of this work.
- **Onboarding builds on a declaration.** The registry generates an attribute's materialization pipeline from what the producer already declared, targeting under two weeks without intensive platform-team support, and every consumer after the first reuses that definition.

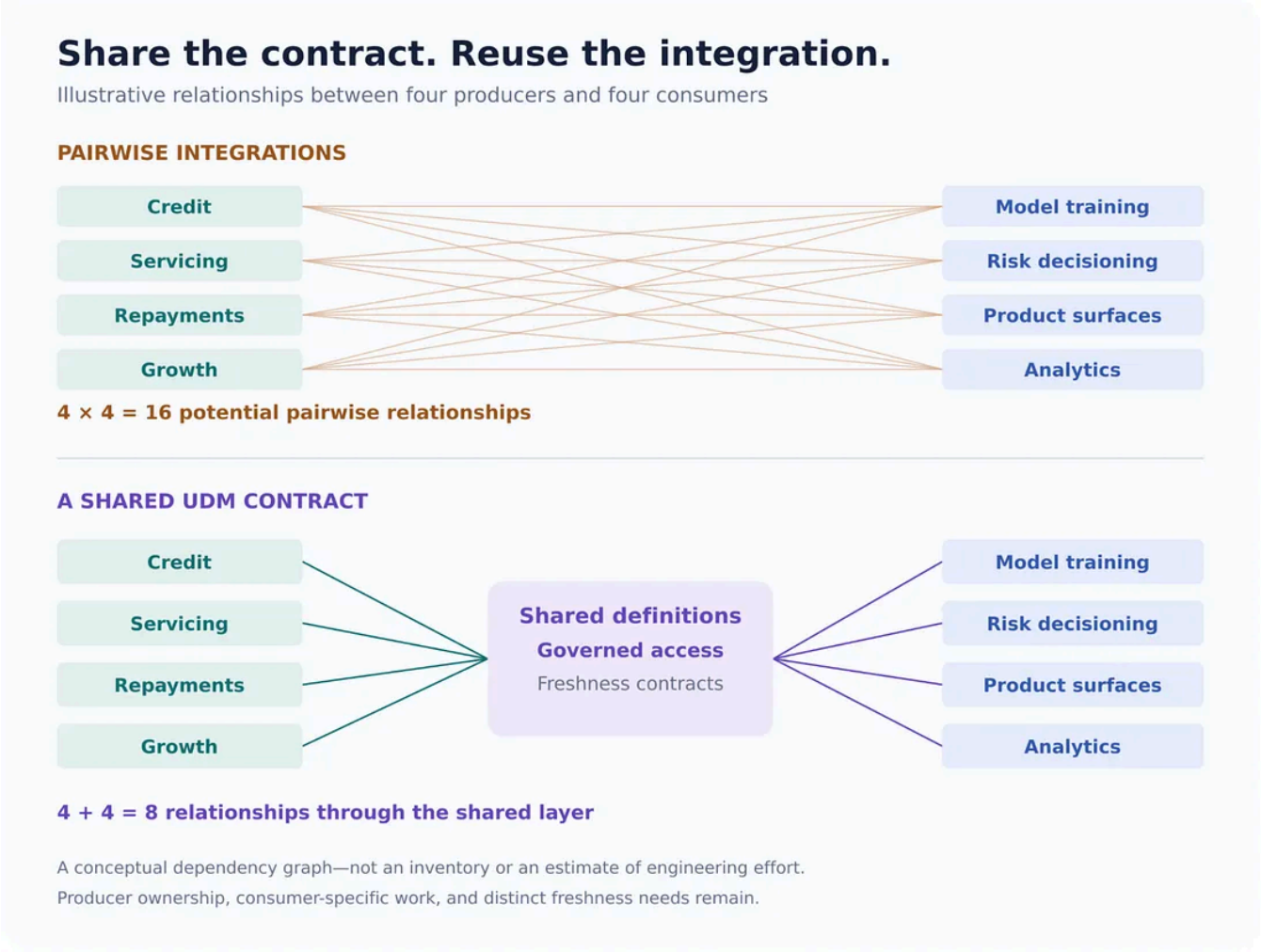


Figure 3. A schematic of contract reuse. Four producers and four consumers can create sixteen pairwise relationships; a shared contract gives each side one integration point. These counts illustrate the shape of the dependency graph, rather than measured implementation effort.

What success looks like

Success is simple to describe. A team starts a new model and the user data is already there. The definitions are the same ones the last team used. They spend their time on the model itself.

It also gets better over time. The better we describe the record, the more a model can learn from it. The more models use it, the more reason each team has to put their data in. The more teams put data in, the more complete the record becomes.

We are writing this because the question is not ours alone. Every company building on AI faces it: what can you learn from the history created through your customer relationships, and is that understanding getting deeper? The answer matters because models are becoming something anyone can buy, and the data foundation behind them is not. When the answer is real, a well-governed data foundation becomes a lasting advantage and the models are how you learn from it.

Affirm's answer is the entire customer lifecycle: how we acquired a new customer, what they were approved for, what they financed, how they paid it back, and what they did next. Repayment is the clearest signal in it today, and every domain that serves a customer adds more. Millions of people, over 14 years, in order. Credit bureaus see the outcome. We see the history behind them.

That is what UDM is for. Today each model rediscovers that history for itself, and the understanding leaves with the project. UDM makes it something the company keeps. Every event we capture and describe well today becomes available to every model authorized to use it.

At checkout, an Affirm model still works with the information available then. The ambition is that "then" keeps getting richer.

Artificial Intelligence + Machine Learning + Data Engineering +
Data Science + Data Architecture +

Follow

Written by Affirm Technology

182 followers · 2 following

Follow

Published in Affirm Tech Blog

905 followers · Last published Oct 22, 2026

www.affirm.com/careers